

1.1 The **epoll** event notification facility

Q1: Does our Mosquitto version use the edge-trigger or level-trigger option of **epoll**?

A: level-trigger

By default, **epoll** operates in level-triggered mode unless the EPOLLET flag is explicitly set.

Reason: Mosquitto needs to ensure that all available data is handled appropriately, especially in high-concurrency environments. With edge-triggered mode, the application must read all available data immediately upon receiving an event, or it risks missing future notifications. This increases complexity and can lead to potential data loss if not managed carefully. Level-triggered mode simplifies the design by repeatedly notifying Mosquitto of the availability of data until the socket buffer is empty, making it more reliable for this type of application.

Evidence: Mosquitto's source code shows that it does not use the EPOLLET flag, so the default configuration leverages level-triggered mode.

Q2: Independent from Mosquitto, use your own words to illustrate a run-time scenario where edge-trigger is better than level-trigger, in the context of using **epoll**.

Edge-triggered mode with **epoll** can be more efficient than level-triggered mode in scenarios where you need high-performance, low-latency handling of I/O events, particularly under heavy load with a large number of file descriptors, like “high-performance web server”.

In a scenario where thousands of connections are active, level-triggered mode would repeatedly notify the server about each connection that has pending data, even if the application is not ready to handle it immediately. This can cause scalability issues, as the server might end up wasting resources managing a large number of repeated events.

In edge-triggered mode, **epoll** will only notify your application once when data becomes available, rather than repeatedly notifying until the data is fully consumed. This means fewer system calls and context switches between the kernel and user space. Under heavy load, this reduction in redundant notifications can significantly improve performance by avoiding unnecessary overhead.

1.2 The broker-side implementation

Give your own reasoning about why the Mosquitto broker only implements a reactive server design and not those more advanced ones?

The introduction of MQTT on Wikipedia: “MQTT is a lightweight, publish-subscribe, machine to machine network protocol for message queue/message queuing service...”, so simplicity and low overload is the advantage of Mosquitto, it is designed to be lightweight and efficient for message brokering, especially in resource-constrained environments like IoT devices. A reactive server design is easier to implement and has lower overhead compared to more advanced patterns like leader/followers.

Practical Scenario: In typical MQTT use cases, clients publish and subscribe to messages with relatively low computational demands. The reactive model, where the server passively reacts to events, fits well because it doesn't require complex orchestration between threads or processes. It also avoids unnecessary complexity that could hinder performance on systems with limited CPU and memory resources, such as embedded devices or Raspberry Pi setups.